FinAssist – Stage 2 Implementation Report

EECS3311 Fall 2026 – Course Project, Stage 2

Repository layout: src/finassist/ (source), build.sh, README.md, docs/diagrams/ (UML diagrams used here). Run: bash build.sh, then java -cp out finassist.Main [gui|cli] [--demo] [--data file].

1. Project overview and implementation status

1.1 Application and users

FinAssist is a personal finance assistant for one individual user (a student or young professional) who wants to record income and expenses, see a monthly picture of their money, follow savings goals and subscriptions, and get plain-language help. It is a Java 11 desktop/command-line program with no external libraries. It has two interchangeable user interfaces (Swing GUIView and text CLIView) that share the same controller, facade and services.

1.2 AI / LLM integration

- **OpenAILLMService** calls the OpenAI chat-completions API through java.net.http (model from OPENAI_MODEL, default gpt-4o-mini; key from OPENAI_API_KEY). Used to word advice, explain unusual expenses, interpret the monthly summary and answer coach questions.
- **VoiceInputService** records 8 s from the microphone and sends it to the OpenAI whisper-1 transcription endpoint. If no key or microphone exists, the view asks the user to type the sentence instead (typed voice fallback).
- **MockLLMService** offline implementation of LLMService used when no key is set and in the self-tests; setAvailable(false) simulates an outage.
- **Design rule** all numbers (totals, profit, savings rate, goal progress, anomaly test, savings recommendation) are computed deterministically in Java. The LLM only produces wording or explanations, so an LLM failure never changes a figure; it only produces a visible notice or error message.

1.3 What works now

- Log income and expenses (GUI form or CLI), by voice (with confirmation) or typed.
- Monthly dashboard: income, expenses, profit, savings rate, category totals, unusual expenses, subscription cost, goals; text bars in CLI and bar chart + progress bar in GUI.
- Monthly summary with AI interpretation (or an explicit notice when the LLM is unavailable).
- Financial goals with progress and required monthly savings; savings advice that automatically switches between goal-based and expense-based strategies.
- Subscriptions: add, remove, list, monthly and annual cost.
- Automatic unusual-expense detection after each saved expense, with an AI explanation.
- AI financial coach for free-form questions, using the user's data as context.
- Saving data between runs (Java serialization, default finassist-data.ser); --demo seeds sample data. 45 automated self-checks (finassist.test.SelfTest) pass via build.sh.

1.4 Limitations and incomplete functionality (honest list)

- Real OpenAI chat and whisper calls and the real microphone were not exercised during development (no API key or microphone in the build environment). They are implemented against the documented API and covered only by the offline mock and typed fallback. The GUI was smoke-tested under a virtual display only.
- Not built from Stage 1: AI explanation of a goal (F06 optional part); explanation of recurring spending and duplicate-subscription warning (F09 / UC09 alternative); coach agent choosing its own tools (F10 uses a fixed context from prepareContext()); LLM-based field extraction from speech (simple English rules are used instead); automatic redraw of an already displayed dashboard (the Observer only invalidates the cache; the user requests the dashboard again); add-transaction shortcuts on an empty dashboard (a message is shown).
- Single user only. The data file uses Java serialization and is not human-readable or versioned.
- Observers run synchronously, so an unusual-expense alert can be printed before the "Expense saved" line.

2. Feature to implementation mapping

Types: Deterministic = computed only by Java code; AI-based = result produced by the LLM; Hybrid = Java computes the facts, the LLM words or explains them. Every request enters through a View, goes to FinancialController, then FinAssistFacade, then services; results return the same way and the controller calls the View.

Video link / timestamp columns are placeholders: the demonstration video must be recorded and linked by the team.

F01 – Voice transaction entry

Feature ID and name	F01 – Voice transaction entry
Stage 1 feature / reference	F01
Status / change	Implemented (modified: typed fallback, confirmation step, rule-based parsing)
Implemented classes / files and responsibilities	VoiceInputService – captures and converts speech; FinAssistFacade – holds the pending entry; FinancialController – asks the user to confirm; GUIView/CLIView – show prompt and take the answer; FinancialService – saves; Income/Expense – result entities.
Key methods	VoiceInputService.captureSpeech(), validateTranscript(), convertToEntry(), setFallbackInput(); FinAssistFacade.logVoiceEntry(), getPendingVoiceEntry(), confirmVoiceEntry(boolean); FinancialController.logVoiceEntry().
Design patterns	MVC, Facade, Observer (after save)
Video link / timestamp	[TO FILL: video link + timestamp]

How the classes collaborate: The user chooses voice entry in the view; the view calls controller.logVoiceEntry(); the facade asks VoiceInputService.captureSpeech() (whisper-1, or typed text if unavailable), validates it (at least two words and a positive amount), converts it to an Income or Expense with regex and keyword rules, and keeps it pending. The controller shows "I understood … Save it? (yes/no)" and reads the answer with view.getUserInput(); confirmVoiceEntry(true) calls FinancialService.addIncome/addExpense, which saves and notifies observers. Invalid text raises ValidationException which the controller shows with displayError.

Behaviour type: Hybrid (AI transcription, deterministic parsing)

F02 – Manual income entry

Feature ID and name	F02 – Manual income entry
Stage 1 feature / reference	F02
Status / change	Implemented

Implemented classes / files and responsibilities	View form/menu; FinancialController; FinAssistFacade; FinancialService; FinancialRepository; Income.
Key methods	FinancialController.logIncome(Income); FinAssistFacade.logIncome(); FinancialService.addIncome(); FinancialRepository.saveEntry(); notifyObservers().
Design patterns	MVC, Facade, Observer
Video link / timestamp	[TO FILL: video link + timestamp]

How the classes collaborate: The view builds an Income (amount, date, source, type) and calls controller.logIncome. The facade forwards to FinancialService.addIncome, which validates (positive amount, date, source), stores it through FinancialRepository.saveEntry and calls notifyObservers(); DashboardService clears its cache. The controller displays "Income saved".

Behaviour type: Deterministic

F03 – Unusual expense detection and explanation

Feature ID and name	F03 – Unusual expense detection and explanation
Stage 1 feature / reference	F03
Status / change	Implemented
Implemented classes / files and responsibilities	FinancialService (subject); ExpenseDetectionAgent (observer and agent); ExpenseDetectionAgentCreator; LLMService; FinancialView.
Key methods	ExpenseDetectionAgent.update(), detectUnusualExpense(Expense), analyzeExpense(Expense), hasEnoughHistory(), setView().
Design patterns	Observer, Factory Method, MVC
Video link / timestamp	[TO FILL: video link + timestamp]

How the classes collaborate: After an expense is saved FinancialService.notifyObservers() calls ExpenseDetectionAgent.update(). The agent takes the newest expense (only once, via lastAnalyzedId), compares it with same-category history (≥3 items, otherwise all expenses, otherwise "not enough history"); it is unusual if amount > mean + 2σ (1.5×mean when σ = 0). If unusual, Expense.setUnusual(true) and analyzeExpense() asks the LLM for an explanation, which is displayed directly on the view. LLM failure shows an error but the expense stays saved.

Behaviour type: Hybrid (deterministic test, AI explanation)

F04 – Manual expense entry

Feature ID and name	F04 – Manual expense entry
Stage 1 feature / reference	F04
Status / change	Implemented
Implemented classes / files and responsibilities	View; FinancialController; FinAssistFacade; FinancialService; FinancialRepository; Expense.
Key methods	FinancialController.logExpense(Expense); FinAssistFacade.logExpense(); FinancialService.addExpense().
Design patterns	MVC, Facade, Observer
Video link / timestamp	[TO FILL: video link + timestamp]

How the classes collaborate: Same path as F02 with an Expense (category, merchant). After saving, observers are notified, which also triggers F03.

Behaviour type: Deterministic

F05 – AI monthly summary / interpretation

Feature ID and name	F05 – AI monthly summary / interpretation
Stage 1 feature / reference	F05
Status / change	Implemented
Implemented classes / files and responsibilities	FinancialController; FinAssistFacade; DashboardService; MonthlySummary; AgentOrchestrator; FinancialCoachAgent; LLMService.
Key methods	FinancialController.requestMonthlySummary(month, year); FinAssistFacade.getMonthlySummary(); DashboardService.generateMonthlyDashboard(); AgentOrchestrator.processRequest(), selectAgent(); FinancialCoachAgent.process(); MonthlySummary.setInterpretation(), setAiNotice().
Design patterns	MVC, Facade, Factory Method (agents created via creators)
Video link / timestamp	[TO FILL: video link + timestamp]

How the classes collaborate: The facade first obtains the deterministic MonthlySummary from DashboardService, returns it directly if empty, otherwise builds a prompt (including which data is missing: goals, subscriptions, income) and sends it through AgentOrchestrator to FinancialCoachAgent and the LLM. The text is stored with setInterpretation(). If LLMUnavailableException occurs, setAiNotice() records that the interpretation is unavailable and the figures stay valid. See SD-03.

Behaviour type: Hybrid

F06 – Financial goal tracking

Feature ID and name	F06 – Financial goal tracking
Stage 1 feature / reference	F06
Status / change	Implemented (AI explanation of goal not built)
Implemented classes / files and responsibilities	FinancialGoal; FinAssistFacade; FinancialRepository; View.
Key methods	FinancialController.addGoal(), requestGoalSummary(); FinAssistFacade.addGoal(), getGoalSummary(); FinancialGoal.getProgress(), getRemainingAmount(), getRequiredMonthlySavings().
Design patterns	MVC, Facade
Video link / timestamp	[TO FILL: video link + timestamp]

How the classes collaborate: addGoal validates name, positive target, non-negative current savings and a future deadline, then calls FinancialRepository.saveGoal. getGoalSummary formats each goal with progress % and required monthly savings = remaining ÷ max(1, ceil(days/30)). All arithmetic is in FinancialGoal.

Behaviour type: Deterministic

F07 – Monthly dashboard

Feature ID and name	F07 – Monthly dashboard
Stage 1 feature / reference	F07
Status / change	Implemented (no automatic redraw)
Implemented classes / files and responsibilities	FinancialController; FinAssistFacade; DashboardService; MonthlySummary; DashboardFormatter; CLIView/GUIView (ChartPanel, JProgressBar).
Key methods	FinancialController.requestDashboard(); FinAssistFacade.getDashboard();

	DashboardService.generateMonthlyDashboard(), calculateIncome(), calculateExpenses(), calculateProfit(), calculateSavingsRate(), update(); MonthlySummary.calculateProfit(), calculateSavingsRate(); FinancialView.showDashboard().
Design patterns	MVC, Facade, Observer
Video link / timestamp	[TO FILL: video link + timestamp]

How the classes collaborate: The month is validated by the facade, DashboardService reads the month's entries from the repository (or its cache), totals them and fills a MonthlySummary, and the controller calls view.showDashboard(summary). DashboardService is an Observer: update() clears the cache so the next request is fresh.

Behaviour type: Deterministic

F08 – Savings advice

Feature ID and name	F08 – Savings advice
Stage 1 feature / reference	F08
Status / change	Implemented
Implemented classes / files and responsibilities	SavingsAnalyzer (Strategy context); SavingsStrategy; GoalBasedSavingsStrategy; ExpenseBasedSavingsStrategy; LLMService.
Key methods	FinancialController.requestSavingsAdvice(); FinAssistFacade.getSavingsAdvice(); SavingsAnalyzer.selectStrategy(), setStrategy(), analyze(), update(); SavingsStrategy.calculateRecommendation().
Design patterns	Strategy, Observer, Facade
Video link / timestamp	[TO FILL: video link + timestamp]

How the classes collaborate: analyze() calls selectStrategy(): GoalBasedSavingsStrategy if goals exist, otherwise ExpenseBasedSavingsStrategy (top 3 categories and the saving from a 10% cut). The chosen strategy computes the recommendation text; the LLM only rephrases it, and if it is unavailable the computed text is returned. As an Observer, SavingsAnalyzer.update() re-selects the strategy when data changes. See SD-03b.

Behaviour type: Hybrid

F09 – Subscription tracking

Feature ID and name	F09 – Subscription tracking
Stage 1 feature / reference	F09
Status / change	Implemented (recurring-spending explanation and duplicate warning not built)
Implemented classes / files and responsibilities	Subscription; SubscriptionService; FinancialRepository; FinAssistFacade; View.
Key methods	FinancialController.manageSubscription(), removeSubscription(), requestSubscriptions(); FinAssistFacade.addSubscription(), removeSubscription(), getSubscriptionSummary(); SubscriptionService.addSubscription(), removeSubscription(), getSubscriptions(), calculateMonthlyCost(); Subscription.getMonthlyCost(), getAnnualCost().
Design patterns	MVC, Facade
Video link / timestamp	[TO FILL: video link + timestamp]

How the classes collaborate: SubscriptionService validates and stores subscriptions; Subscription converts weekly/monthly/yearly prices to monthly and annual cost. The facade builds a summary with the total and the controller displays it.

Behaviour type: Deterministic

F10 – AI financial coach

Feature ID and name	F10 – AI financial coach
Stage 1 feature / reference	F10
Status / change	Implemented (fixed context, no agent-chosen tools)
Implemented classes / files and responsibilities	FinancialCoachAgent; FinancialCoachAgentCreator; AgentOrchestrator; LLMService; FinancialRepository.
Key methods	FinancialController.askFinancialCoach(String); FinAssistFacade.askCoach(); AgentOrchestrator.processRequest(), selectAgent(), addAgent(); FinancialCoachAgent.process(), generateAdvice(), prepareContext().
Design patterns	Factory Method, Facade, MVC
Video link / timestamp	[TO FILL: video link + timestamp]

How the classes collaborate: The orchestrator routes requests starting with "analyze unusual expense" to ExpenseDetectionAgent and every other request to FinancialCoachAgent, creating the agent through its creator if absent. The coach asks a clarifying question when the question has fewer than two words, asks the user to add data when there is none, otherwise builds a context (totals, categories, goals, last 6 conversation lines) with prepareContext() and sends it to the LLM.

Behaviour type: AI-based

N1 – Saved data between sessions

Feature ID and name	N1 – Saved data between sessions
Stage 1 feature / reference	New
Status / change	New
Implemented classes / files and responsibilities	FinancialRepository (optional file), Main (--data).
Key methods	FinancialRepository.persist(), saveEntry(), saveGoal(), saveSubscription().
Design patterns	N/A (supports MVC Model)
Video link / timestamp	[TO FILL: video link + timestamp]

How the classes collaborate: Each save method writes the User object to the file with Java serialization; Main loads it at start-up.

Behaviour type: Deterministic

N2 – Offline AI mode and typed voice fallback

Feature ID and name	N2 – Offline AI mode and typed voice fallback
Stage 1 feature / reference	New
Status / change	New
Implemented classes / files and responsibilities	MockLLMService; LLMService; VoiceInputService; Main.
Key methods	MockLLMService.generate(), setAvailable(); VoiceInputService.setFallbackInput().
Design patterns	N/A (LLMService interface allows substitution)
Video link / timestamp	[TO FILL: video link + timestamp]

How the classes collaborate: Main chooses OpenAILLMService when OPENAI_API_KEY exists, otherwise MockLLMService, so the whole program and the tests run without network.

Behaviour type: Deterministic

N3 – Complete command-line interface

Feature ID and name	N3 – Complete command-line interface
Stage 1 feature / reference	New
Status / change	New (Stage 1 only described one interface)
Implemented classes / files and responsibilities	CLIView; FinancialView.
Key methods	CLIView.run(), display(), displayError(), showDashboard(), getUserInput().
Design patterns	MVC (second View)
Video link / timestamp	[TO FILL: video link + timestamp]

How the classes collaborate: A 12-option text menu calls the same controller methods as the GUI.

Behaviour type: Deterministic

N4 – Background GUI actions

Feature ID and name	N4 – Background GUI actions
Stage 1 feature / reference	New
Status / change	New
Implemented classes / files and responsibilities	GUIView.
Key methods	GUIView worker (single-thread executor), invokeLater, submitEntry().
Design patterns	MVC
Video link / timestamp	[TO FILL: video link + timestamp]

How the classes collaborate: GUIView runs controller calls on a worker thread and updates Swing components with invokeLater so the window never freezes during LLM or microphone calls.

Behaviour type: Deterministic

3. Implemented design patterns

All five patterns required by the course are implemented and match the Stage 1 design. The class diagram below was generated from the compiled code (javap), so every class, attribute and method shown exists in the source. Colours: blue MVC, green Facade, pink Strategy, light green Observer, purple Factory Method, beige entities, grey infrastructure.

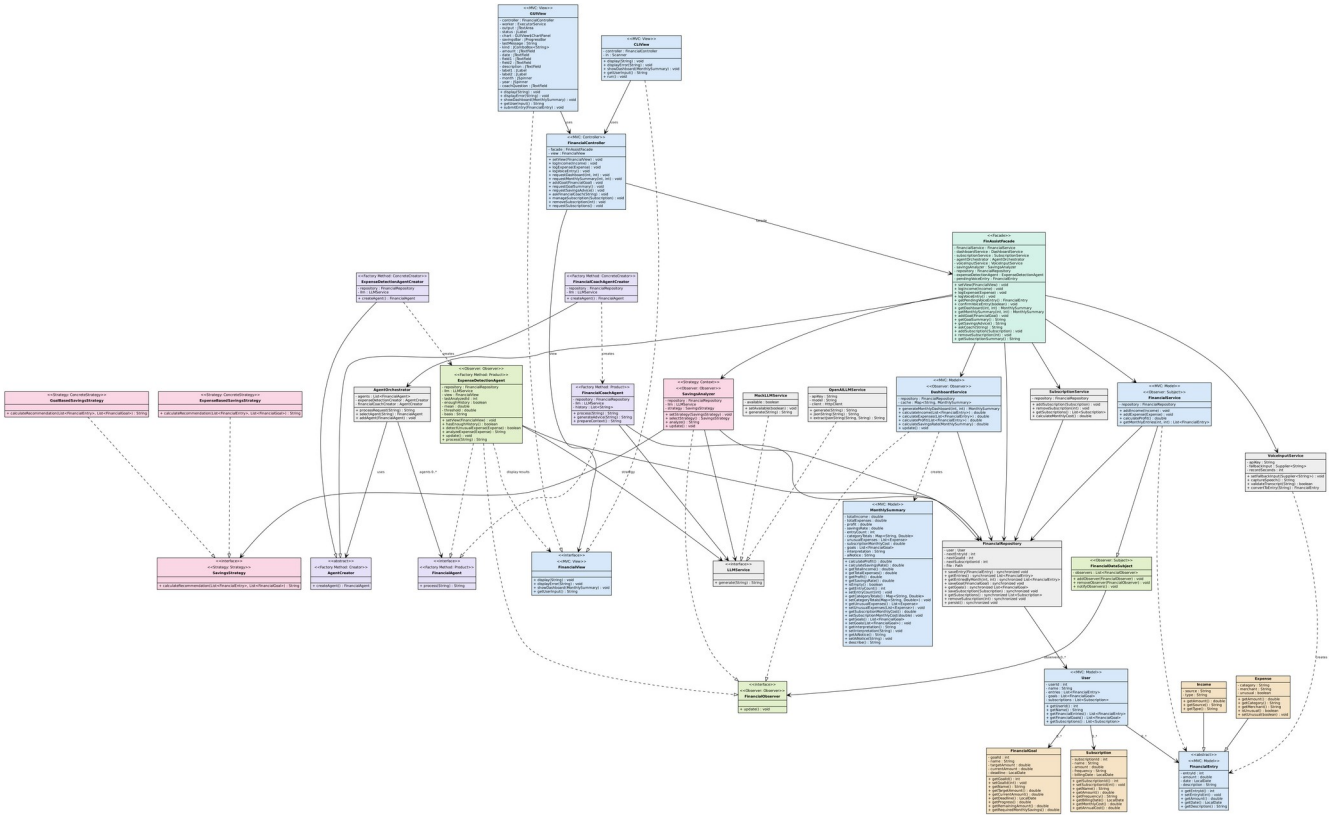


Figure 1 – Updated UML class diagram of the implemented system (also available as docs/diagrams/class_diagram_full.svg).

3.1 MVC (Model–View–Controller)

Design problem	Separate the user interface from business logic so that two different interfaces (GUI and CLI) can use the same logic.
Participating classes and roles	View: FinancialView (interface), GUIView, CLIView, DashboardFormatter. Controller: FinancialController. Model: FinancialService, DashboardService, SubscriptionService, SavingsAnalyzer, FinancialRepository, User, FinancialEntry, Income, Expense, FinancialGoal, Subscription, MonthlySummary.
Source files and methods	view/FinancialView.java, GUIView.java, CLIView.java; controller/FinancialController.java (logIncome, logExpense, logVoiceEntry, requestDashboard, requestMonthlySummary, addGoal, requestGoalSummary, requestSavingsAdvice, askFinancialCoach, manageSubscription, removeSubscription, requestSubscriptions, setView); view methods display, displayError, showDashboard, getUserInput.
How the implementation follows the pattern / why useful	Views never touch services: they only call the controller. The controller never formats output: it calls FinancialView. Its private run() converts ValidationException, LLMUnavailableException and RuntimeException into displayError. Useful because GUIView and CLIView are interchangeable (Main selects gui|cli).

Match with Stage 1	Matches Stage 1. Added: FinancialView.showDashboard() and getUserInput() (needed for the confirmation dialog and chart), and the controller → view association.

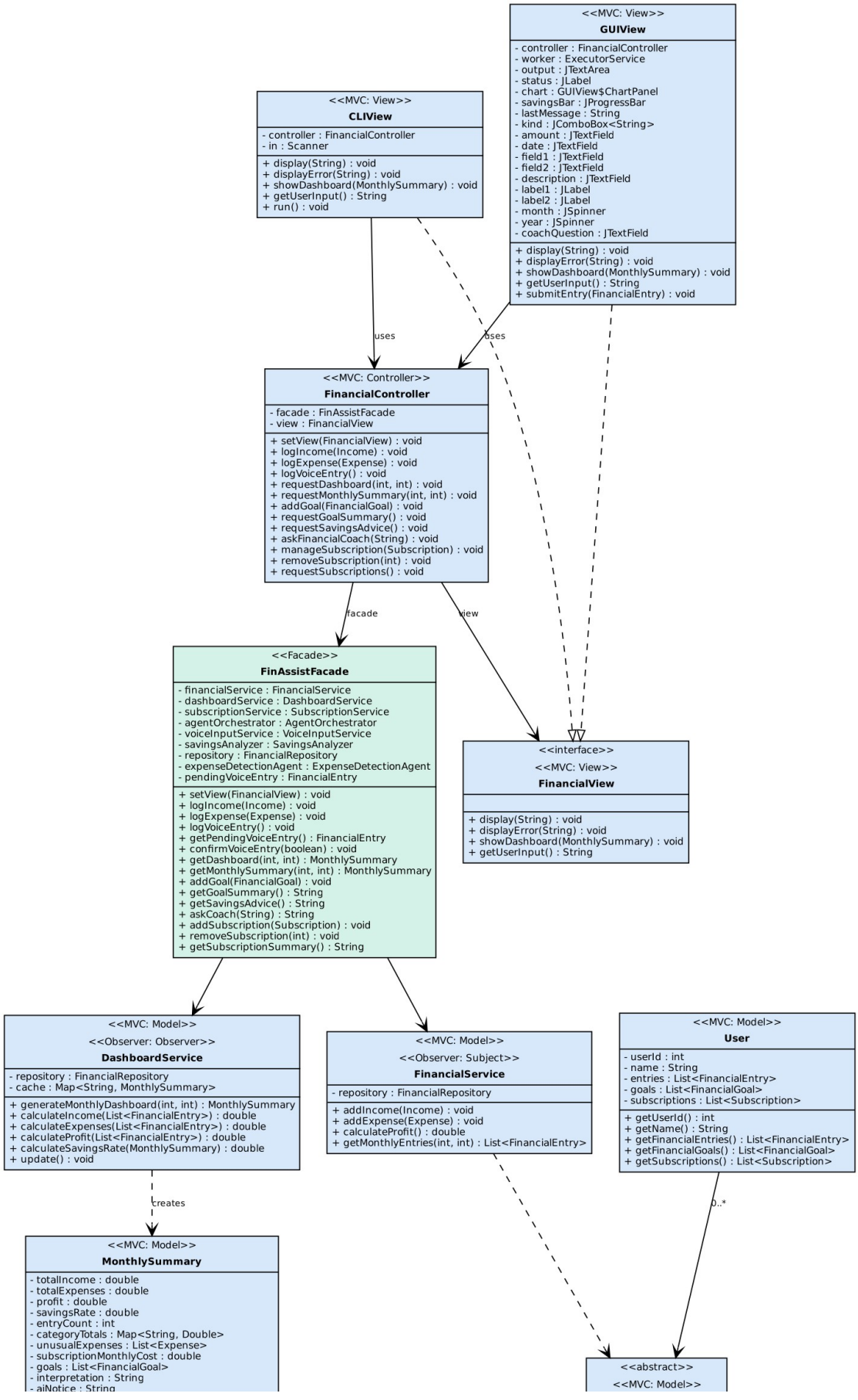

<<MVC: View>>
GUIView
- controller : FinancialController
- worker : ExecutorService
- output : JTextArea
- status : JLabel
- chart : GUIView$ChartPanel
- savingsBar : JProgressBar
- lastMessage : String
- kind : JComboBox<String>
- amount : JTextField
- date : JTextField
- field1 : JTextField
- field2 : JTextField
- description : JTextField
- label1 : JLabel
- label2 : JLabel
- month : JSpinner
- year : JSpinner
- coachQuestion : JTextField
+ display(String) : void
+ displayError(String) : void
+ showDashboard(MonthlySummary) : void
+ getUserInput() : String
+ submitEntry(FinancialEntry) : void
<<MVC: View>>
CLIView
- controller : FinancialController
- in : Scanner
+ display(String) : void
+ displayError(String) : void
+ showDashboard(MonthlySummary) : void
+ getUserInput() : String
+ run() : void
uses
uses
<<MVC: Controller>>
FinancialController
- facade : FinAssistFacade
- view : FinancialView
+ setView(FinancialView) : void
+ logIncome(Income) : void
+ logExpense(Expense) : void
+ logVoiceEntry() : void
+ requestDashboard(int, int) : void
+ requestMonthlySummary(int, int) : void
+ addGoal(FinancialGoal) : void
+ requestGoalSummary() : void
+ requestSavingsAdvice() : void
+ askFinancialCoach(String) : void
+ manageSubscription(Subscription) : void
+ removeSubscription(int) : void
+ requestSubscriptions() : void
facade
view
<<Facade>>
FinAssistFacade
- financialService : FinancialService
- dashboardService : DashboardService
- subscriptionService : SubscriptionService
- agentOrchestrator : AgentOrchestrator
- voiceInputService : VoiceInputService
- savingsAnalyzer : SavingsAnalyzer
- repository : FinancialRepository
- expenseDetectionAgent : ExpenseDetectionAgent
- pendingVoiceEntry : FinancialEntry
+ setView(FinancialView) : void
+ logIncome(Income) : void
+ logExpense(Expense) : void
+ logVoiceEntry() : void
+ getPendingVoiceEntry() : FinancialEntry
+ confirmVoiceEntry(boolean) : void
+ getDashboard(int, int) : MonthlySummary
+ getMonthlySummary(int, int) : MonthlySummary
+ addGoal(FinancialGoal) : void
+ getGoalSummary() : String
+ getSavingsAdvice() : String
+ askCoach(String) : String
+ addSubscription(Subscription) : void
+ removeSubscription(int) : void
+ getSubscriptionSummary() : String
<<interface>>
<<MVC: View>>
FinancialView
+ display(String) : void
+ displayError(String) : void
+ showDashboard(MonthlySummary) : void
+ getUserInput() : String
<<MVC: Model>>
<<Observer: Observer>>
DashboardService
- repository : FinancialRepository
- cache : Map<String, MonthlySummary>
+ generateMonthlyDashboard(int, int) : MonthlySummary
+ calculateIncome(List<FinancialEntry>) : double
+ calculateExpenses(List<FinancialEntry>) : double
+ calculateProfit(List<FinancialEntry>) : double
+ calculateSavingsRate(MonthlySummary) : double
+ update() : void
<<MVC: Model>>
<<Observer: Subject>>
FinancialService
- repository : FinancialRepository
+ addIncome(Income) : void
+ addExpense(Expense) : void
+ calculateProfit() : double
+ getMonthlyEntries(int, int) : List<FinancialEntry>
<<MVC: Model>>
User
- userId : int
- name : String
- entries : List<FinancialEntry>
- goals : List<FinancialGoal>
- subscriptions : List<Subscription>
+ getUserId() : int
+ getName() : String
+ getFinancialEntries() : List<FinancialEntry>
+ getFinancialGoals() : List<FinancialGoal>
+ getSubscriptions() : List<Subscription>
creates
0..*
<<MVC: Model>>
MonthlySummary
- totalIncome : double
- totalExpenses : double
- profit : double
- savingsRate : double
- entryCount : int
- categoryTotals : Map<String, Double>
- unusualExpenses : List<Expense>
- subscriptionMonthlyCost : double
- goals : List<FinancialGoal>
- interpretation : String
- aiNotice : String
<<abstract>>
<<MVC: Model>>

Figure – MVC (Model–View–Controller) classes (extract of the implemented class diagram).

3.2 Facade

Design problem	Give the controller one simple entry point instead of knowledge of ten collaborating services and agents.
Participating classes and roles	Facade: FinAssistFacade. Subsystem: FinancialService, DashboardService, SubscriptionService, SavingsAnalyzer, VoiceInputService, AgentOrchestrator, FinancialRepository.
Source files and methods	facade/FinAssistFacade.java: constructor wiring; logIncome, logExpense, logVoiceEntry, getPendingVoiceEntry, confirmVoiceEntry, getDashboard, getMonthlySummary, addGoal, getGoalSummary, getSavingsAdvice, askCoach, addSubscription, removeSubscription, getSubscriptionSummary, setView.
How the implementation follows the pattern / why useful	The controller holds only a FinAssistFacade. The facade creates the services, creators and orchestrator, registers the observers on FinancialService and applies shared validation (month range, goal fields). Useful: changing a service does not affect the controller or the views.
Match with Stage 1	Matches Stage 1. Added facade methods addGoal, getMonthlySummary, getSubscriptionSummary, getPendingVoiceEntry, confirmVoiceEntry, setView.

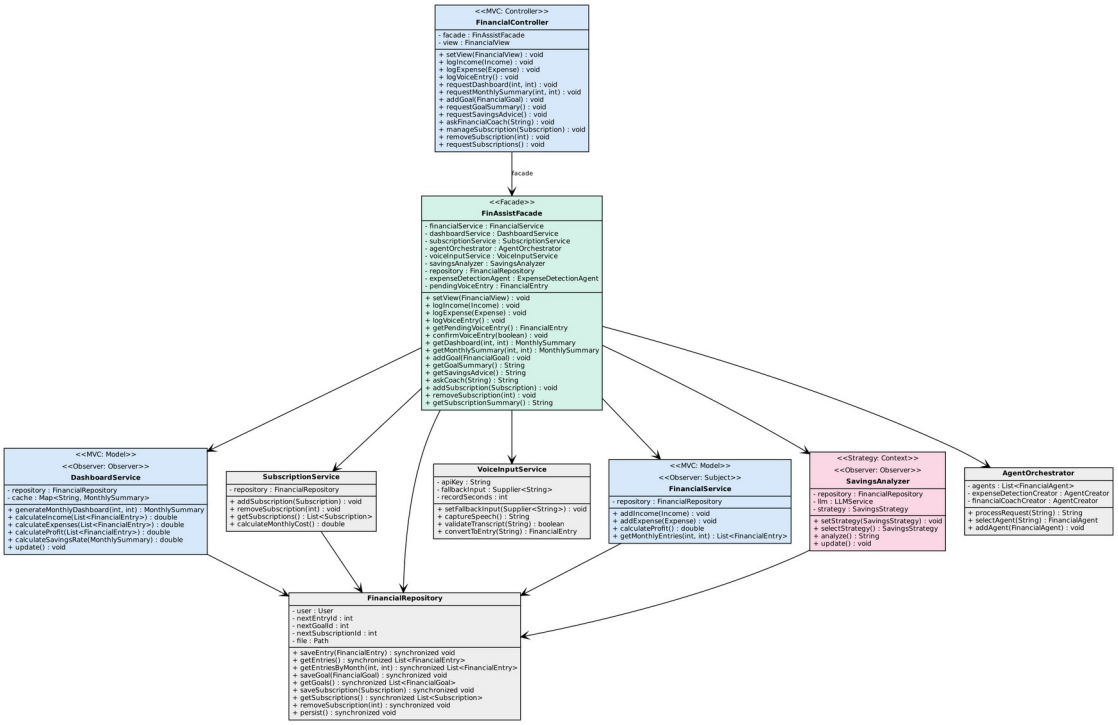


Figure – Facade classes (extract of the implemented class diagram).

3.3 Strategy

Design problem	Choose the savings calculation at run time (goal-based vs expense-based) without if/else in the analyzer.
Participating classes and roles	Context: SavingsAnalyzer. Strategy: SavingsStrategy. Concrete strategies: GoalBasedSavingsStrategy, ExpenseBasedSavingsStrategy (package-private helper Stats).
Source files and methods	service/SavingsAnalyzer.java: setStrategy(), selectStrategy(),

	analyze(), update(); strategy/*.java: calculateRecommendation(entries, goals).
How the implementation follows the pattern / why useful	The analyzer delegates the calculation to whichever SavingsStrategy is set; selectStrategy() picks GoalBased when goals exist. New strategies can be added without changing the analyzer.
Match with Stage 1	Matches Stage 1.

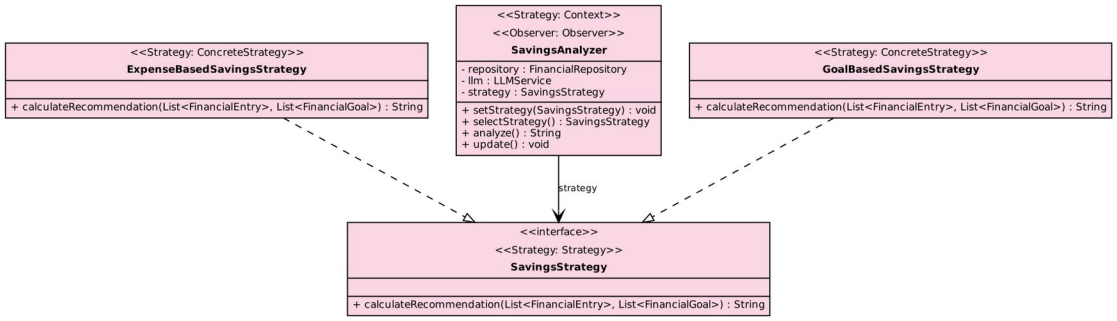


Figure – Strategy classes (extract of the implemented class diagram).

3.4 Observer

Design problem	When financial data changes, dependent components must refresh or react without FinancialService knowing about them.
Participating classes and roles	Subject: FinancialDataSubject (base class of FinancialService). Observer: FinancialObserver. Concrete observers: DashboardService, SavingsAnalyzer, ExpenseDetectionAgent.
Source files and methods	observer/FinancialDataSubject.java: addObserver, removeObserver, notifyObservers; FinancialService.addIncome/addExpense call notifyObservers(); update() in the three observers.
How the implementation follows the pattern / why useful	After every save the subject calls update() on all observers: DashboardService clears its cache, SavingsAnalyzer re-selects its strategy, ExpenseDetectionAgent checks the newest expense. FinAssistFacade registers them.
Match with Stage 1	Matches Stage 1. Note: notifications are synchronous.

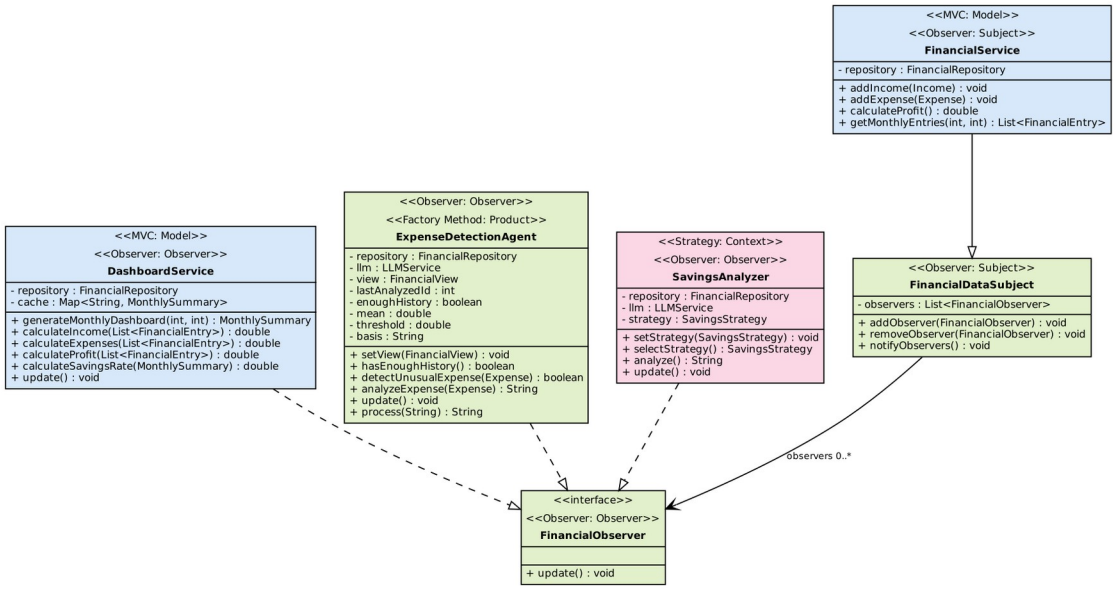


Figure – Observer classes (extract of the implemented class diagram).

3.5 Factory Method

Design problem	Create agents without the orchestrator depending on concrete agent classes.
Participating classes and roles	Creator: AgentCreator (abstract). Concrete creators: ExpenseDetectionAgentCreator, FinancialCoachAgentCreator. Product: FinancialAgent. Concrete products: ExpenseDetectionAgent, FinancialCoachAgent. Client: AgentOrchestrator.
Source files and methods	agent/AgentCreator.java createAgent(); the two creators override createAgent(); AgentOrchestrator.selectAgent(), processRequest(), addAgent().
How the implementation follows the pattern / why useful	selectAgent() finds an existing agent or asks the matching creator to createAgent(), so adding a new agent type means adding one creator. Useful for the planned extension of more AI agents.
Match with Stage 1	Matches Stage 1 (creator constructors take repository and LLMService).

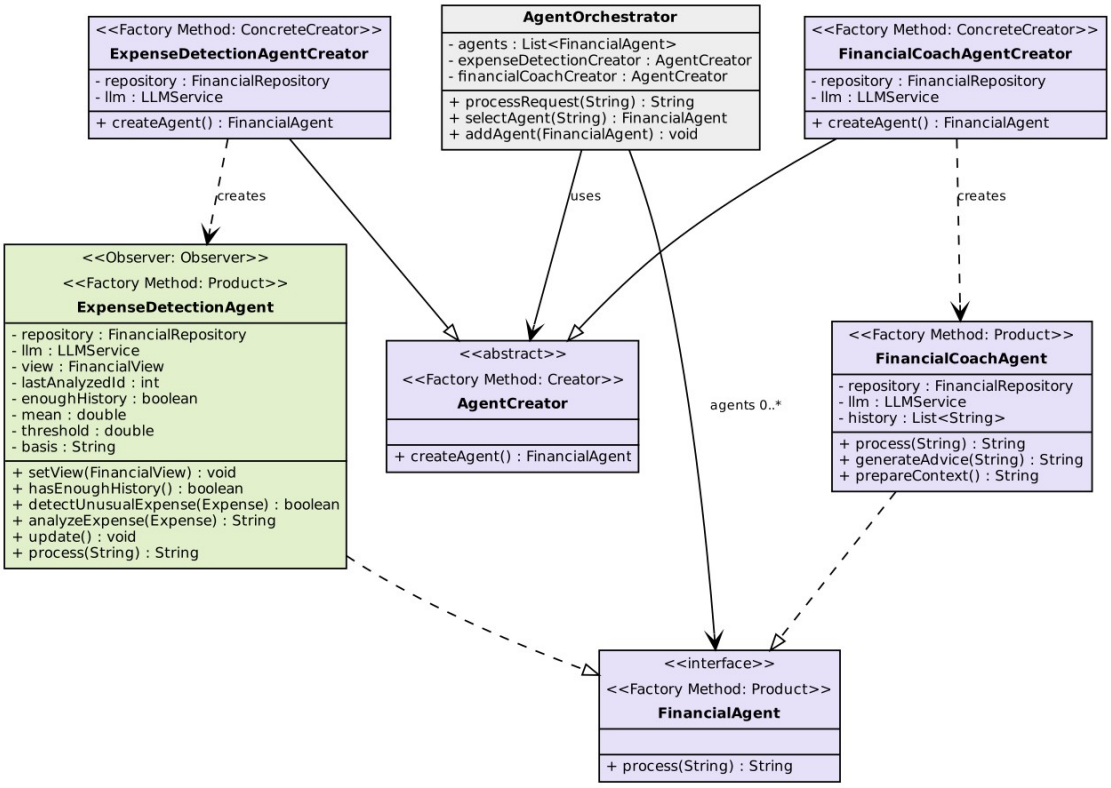


Figure – Factory Method classes (extract of the implemented class diagram).

3.6 Updated sequence diagrams

The Stage 1 use-case diagram and sequence diagrams SD-02 (unusual expense), SD-04 (subscriptions) and SD-05 (coach) remain valid with these remarks: in SD-02 the agent displays the alert on the view itself before the "saved" message; SD-04 additionally offers remove and list. SD-01 and SD-03 changed and are redrawn from the code:

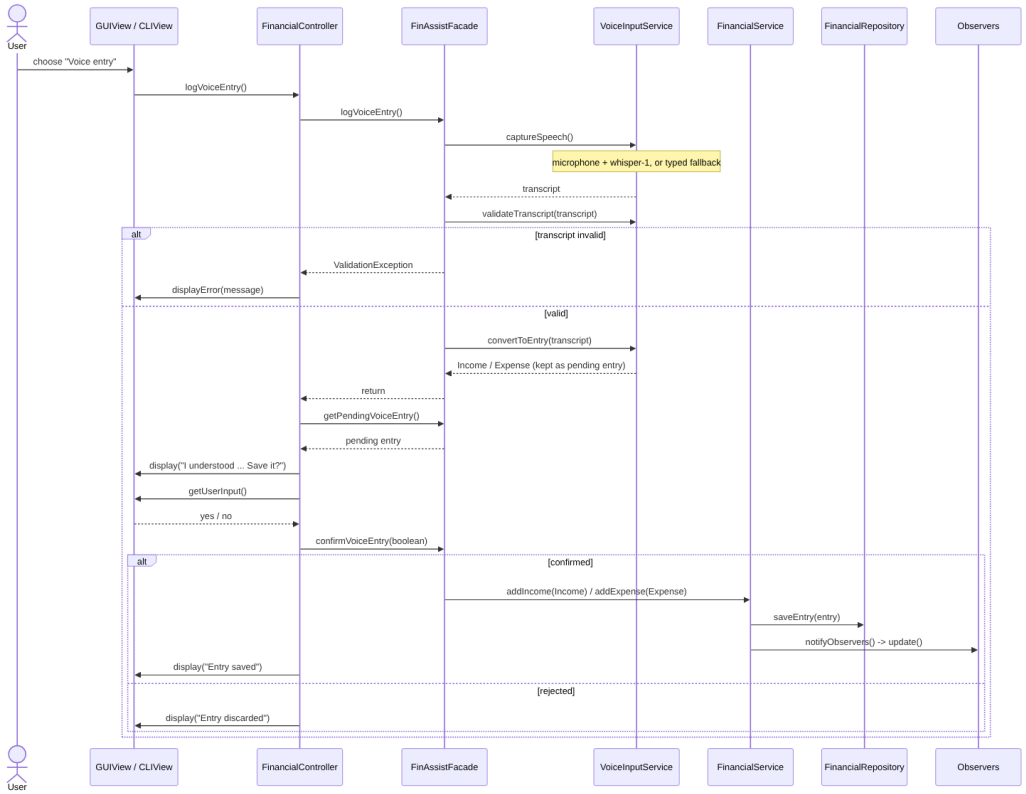


Figure – SD-01 Voice entry as implemented (confirmation step, typed fallback).

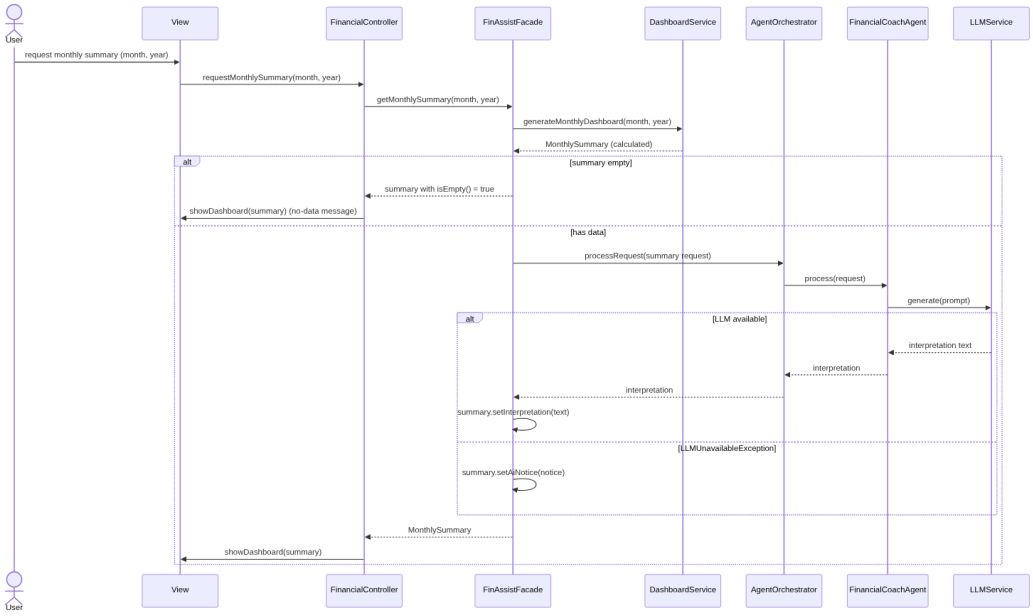


Figure – SD-03 Monthly summary as implemented (getMonthlySummary via orchestrator and coach).

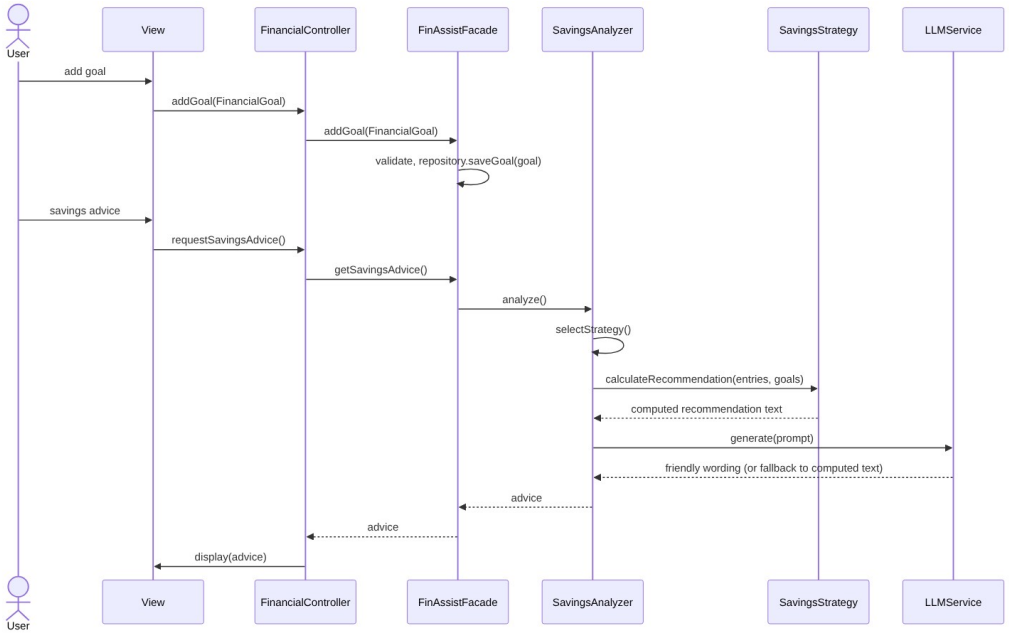


Figure – Goal tracking and savings advice (Strategy) as implemented.

4. Changes and rationale

Format: Original design → Implemented change → Reason → Impact.

Original design	Implemented change	Reason	Impact
F01 voice entry: speech converted straight to a transaction by the LLM	Whisper transcription (or typed fallback), rule-based parsing, user confirmation before saving; new facade methods getPendingVoiceEntry, confirmVoiceEntry	Parsing errors would silently store wrong money values; rules are testable and work offline; typed fallback keeps the feature usable without a microphone/key	Safer and testable; limited to simple English sentences
F06 goal with AI explanation	Deterministic progress and required savings only	Explanation was optional; numbers are the user need	Less AI; no behaviour risk
F07 dashboard redraws automatically on data change	Observer clears DashboardService cache; user requests dashboard again	Keeps Observer meaningful without UI threading complexity	Slightly less convenient; always correct data
F09 recurring-spending explanation and duplicate warning	Not implemented; add/remove/list/cost implemented	Time; core tracking need covered	Reduced scope
F10 coach selects tools	Fixed context via prepareContext() plus recent history	Simpler, deterministic and cheaper; avoids unpredictable tool calls	Coach cannot fetch extra data on demand
F05 summary generated only by AI	Deterministic MonthlySummary plus AI interpretation, notice if LLM unavailable	Figures must never depend on the LLM	Hybrid design; clear failure behaviour
One user interface	GUIView and CLIView behind FinancialView (N3, N4)	Course asked for GUI and CLI; demonstrates MVC	Two views to maintain
Data in memory only	Serialization file (N1)	Users expect data to remain	Not human-readable
Only the real LLM	LLMService interface with OpenAILLMService and MockLLMService (N2)	Tests and demos without network or key	Real API calls not tested here
Observer: DashboardService	SavingsAnalyzer also an	Strategy stays correct after	Pattern participants increased

and agent observers	observer (re-selects strategy)	goals/entries change	
Facade/controller methods in Stage 1 list	Added addGoal, getMonthlySummary, getSubscriptionSummary, removeSubscription, requestSubscriptions, setView and others	Needed by the views for features F05, F06, F09	Class diagram and sequence diagrams updated
No pattern replaced	All five patterns kept as designed	Stage 1 choices fit the implementation	No architectural rework

5. AI tools and collaboration log

Tool	LLM / model	Purpose	Example tasks
Claude (Claude Code agent, claude.ai)	claude-sonnet-5-5 (configured; the serving model may differ and cannot be confirmed)	Design, diagrams, implementation, testing, documentation	UMLet use-case, class and sequence diagrams; consistency audit; the whole Java implementation and 45 self-checks; this report
OpenAI API (runtime component of FinAssist)	gpt-4o-mini (default) and whisper-1	Product feature, not a development tool	Wording of advice, explanations, coach, summary interpretation; speech transcription. Not exercised during development

Token cost and money cost: [TO FILL from your Claude usage page – not available to the assistant, so no figure is invented here]. OpenAI runtime cost during development: none (no API calls were made).

Representative interactions

- **Diagrams** asked for use-case and sequence diagrams in UMLet; Claude generated .uxf files with scripts. After the instruction "don't make it complicated", the sequence diagrams were simplified.
- **Consistency** asked that classes and methods match across diagrams. An audit found misaligned use-case relations, duplicate or missing use-case descriptions, different creator names and missing methods; they were fixed and the Word document updated.
- **Patterns** asked to include MVC, Facade, Strategy, Observer and Factory Method; Claude added stereotypes and a legend to the class diagram.
- **Implementation** asked to create the project from the document with exact patterns, classes and features; Claude wrote the Java code and SelfTest (45 checks).
- **Debugging examples** the voice parser captured too much text for the merchant (fixed by limiting the regex to "at|from"); the orchestrator routed unusual-expense requests to the wrong agent (fixed with startsWith("analyze unusual expense")).
- **Human contribution** the team defined requirements, features, patterns and checks, reviewed every output, and chose to keep deterministic calculations separate from AI wording. The team must review and run the code before submission.

6 Development and verification summary

6.1 Weekly commit summary

The commit history lives in your GitHub repository, which the assistant cannot access, so the weekly rows below must be completed from git log. Run: git log --since="<week start>" --until="<week end>" --pretty="%ad %s" --date=short. The "Work done" column lists the actual development phases, which you can match to your commits. No commit counts or dates are invented here.

Week	Commits (from git log)	Work done
Stage 1 weeks	[TO FILL]	Requirements, use-case diagram, sequence diagrams SD-01 to SD-05, class diagram with the five patterns, traceability and consistency audit, Stage 1 document aligned to the diagrams.
Week of 2026-11-02	[TO FILL]	GUIView and CLIView design on top of the MVC structure (request of 2026-11-06).
Week of 2026-11-02 to 11-08	[TO FILL]	Java implementation of all classes (2026-11-07), 45 self-checks, build.sh and README, change log and AI-collaboration document, then this report with the regenerated class diagram and redrawn sequence diagrams (2026-11-08).

6.2 Checks performed

- **Automated self-checks** build.sh compiles every class and runs finassist.test.SelfTest with assertions enabled; the final run printed "All 45 checks passed." The checks use MockLLMService and a recording view, so no network is needed.
- **Feature checks** F01 transcript validation, conversion of spending and income sentences, confirmation prompt, saving after confirmation, unclear speech; F02/F04 invalid amount and missing category rejected, income and expense saved; F03 insufficient-history message, alert with AI explanation, no alert for normal expense, LLM outage message; F05 interpretation text and the no-LLM notice; F06 expired deadline rejected, progress, remaining amount, required monthly savings; F07 income, expense total, unusual list, category totals, empty-data message, month validation; F08 advice uses the goal strategy; F09 monthly total, invalid subscription, removing a missing one, annual cost; F10 clarifying question, coach answer, no-data prompt.
- **Pattern checks** Strategy selection (goals give GoalBasedSavingsStrategy, none give ExpenseBasedSavingsStrategy); Factory Method (orchestrator creates the right agent and reuses it, creators extend AgentCreator); Observer (add, notify, remove; FinancialService is the subject; DashboardService and SavingsAnalyzer are observers; dashboard refreshes after a new entry); MVC/Facade exercised through the controller on every feature test.
- **Consistency checks** the class diagram in this report is generated from javap output of the compiled code; the sequence diagrams SD-01 and SD-03 were checked against FinAssistFacade and FinancialController source; earlier audit compared use-case, class and sequence diagrams and the Stage 1 document.
- **Manual smoke test** the Swing GUI was started under a virtual display (xvfb) and a screenshot taken; the CLI menu was run with --demo.

6.3 Important defects found and corrected

Defect	Found by	Fix
Voice parser captured too much text as the merchant (for example whole	Voice self-check on "coffee at Tim Hortons"	Merchant regex restricted to the words "at" and "from"

phrases)		
Unusual-expense requests (F05 flow) were routed to the wrong agent	Factory Method / orchestrator check	AgentOrchestrator.selectAgent() routes only requests starting with "analyze unusual expense" to ExpenseDetectionAgent, all others to the coach
Test harness did not feed typed voice input correctly	F01 confirmation check	Voice input queue in SelfTest corrected
build.sh could not run directly (bad interpreter: Permission denied on the output mount)	First build attempt	README now says: bash build.sh
Early sequence diagrams used methods not in the class diagram; use-case relations misaligned; duplicate or missing use-case text; creator names differed	Consistency audit (Stage 1 phase)	Diagrams rebuilt strictly from the class diagram and the Stage 1 document corrected

6.4 Remaining limitations

These repeat and extend section 1.4:

- Real OpenAI chat and whisper calls and the physical microphone are untested; only the offline mock and typed fallback were verified.
- Not built: AI goal explanation, recurring-spending explanation, duplicate-subscription warning, agent-chosen tools, LLM-based speech field extraction, automatic dashboard redraw, add-transaction shortcuts on an empty dashboard.
- Voice parsing supports only simple English sentences with an amount.
- GUI was smoke-tested only; no automated GUI tests. Observers run synchronously.
- One user; serialized data file is not human-readable or versioned; no file-corruption recovery.
- Video links, token cost, money cost and weekly commit rows still need the team's own data.